

A-Z BUILD PROMPT: REMOTE GUIDANCE AND COUNSELING PLATFORM

Act as a senior product designer, software architect, frontend engineer, backend engineer, DevOps engineer, security engineer, and QA engineer.

Build a complete production-oriented website called **CounselConnect**. The name must be configurable and easy to replace.

CounselConnect helps people:

- Find verified guidance and counseling professionals.
- Book private counseling sessions.
- Conduct one-to-one video or voice calls.
- Host private or public group discussions.
- Join scheduled support groups.
- Exchange secure messages.
- Share approved counseling resources.
- Receive session reminders.
- Report abuse or safety concerns.
- Manage counselors, groups, users, and sessions from an administrator dashboard.

The platform must work on desktop, tablet, and mobile devices.

1. Critical product boundaries

This platform is not automatically an emergency service.

Display a visible emergency notice during onboarding, booking, and sessions:

"If you or another person is in immediate danger, contact the appropriate local emergency service. This platform is not an emergency response service."

Do not allow the system or an AI feature to:

- Diagnose mental-health conditions.
- Prescribe medication.
- Claim to replace licensed professionals.
- Produce automatic clinical decisions.
- Pretend that unverified hosts are licensed counselors.

Separate these account types clearly:

- Licensed or professionally verified counselor.

- School or career guidance counselor.
- Peer-support facilitator.
- Group moderator.
- General participant.

Only verified professionals may display a “Verified Professional” badge.

2. Delivery strategy

Build the platform in three controlled stages.

Stage 1: Production MVP

Implement:

- Authentication.
- User profiles.
- Counselor verification workflow.
- Counselor directory and search.
- Availability calendar.
- Appointment booking.
- One-to-one voice and video sessions.
- Group discussions.
- Waiting room.
- In-call text chat.
- Screen sharing.
- Host moderation controls.
- Notifications.
- Session notes.
- Safety reporting.
- Administrator dashboard.
- Privacy controls.
- Docker development environment.
- Automated tests.

Stage 2: Operational improvements

Implement:

- Recurring group meetings.
- Calendar integration.
- Optional recording with explicit consent.
- Counselor analytics.
- Resource library.
- Organization accounts for schools, NGOs, churches, clinics, and companies.
- Multi-language interface.
- Improved low-bandwidth support.

Stage 3: Optional advanced functions

Prepare extension points for:

- Live captions.
- Translation.
- AI-assisted session summaries.
- Mobile applications.
- Telephony dial-in.
- Breakout rooms.
- Collaborative whiteboard.

Do not implement AI diagnosis or automatic risk scoring.

3. Required technology stack

Use a TypeScript monorepo.

Frontend

- Current stable Next.js with App Router.
- React.
- TypeScript in strict mode.
- Tailwind CSS.
- shadcn/ui and Radix UI.
- TanStack Query.
- React Hook Form.
- Zod validation.
- Lucide icons.
- Motion for restrained interface animation.
- LiveKit React components and client SDK.

Backend

- Node.js LTS.
- NestJS.
- REST API with an OpenAPI specification.
- PostgreSQL.
- Prisma ORM.
- Redis.
- BullMQ for reminders and background jobs.
- LiveKit for voice and video infrastructure.
- S3-compatible storage for approved uploads and recordings.
- SMTP-compatible email provider.
- Optional SMS and WhatsApp notification adapters.

Testing and operations

- Vitest for unit tests.
- Supertest for API integration tests.
- Playwright for end-to-end tests.
- axe accessibility testing.
- Docker and Docker Compose.
- GitHub Actions-compatible CI configuration.
- OpenTelemetry-compatible tracing.
- Structured JSON logs.
- Health and readiness endpoints.

Use stable compatible dependency versions and commit the lockfile.

4. Repository structure

Create:

- `apps/web` – Next.js frontend.
- `apps/api` – NestJS API.
- `apps/worker` – background jobs and reminders.
- `packages/ui` – shared UI components.
- `packages/types` – shared TypeScript types.
- `packages/validation` – shared Zod schemas.
- `packages/database` – Prisma schema, migrations, and seed data.
- `packages/config` – validated environment configuration.
- `infra/docker` – Docker support.
- `infra/livekit` – local LiveKit configuration.
- `docs` – setup, architecture, API, security, deployment, and testing guides.

Use pnpm workspaces. Provide one-command local startup.

5. User roles and permissions

Implement strict role-based access control.

Guest

Can:

- View the landing page.
- Browse public counselors.
- View public groups.
- Read public resources.
- Register or sign in.

Client or participant

Can:

- Manage their profile.
- Search for counselors.
- Book appointments.
- Join authorized meetings.
- Send messages in authorized conversations.
- Join or leave groups.
- Give or withdraw recording consent.
- Download their permitted data.
- Submit reports.
- Request account deletion.

Counselor

Can:

- Create a professional profile.
- Submit credentials for verification.
- Set availability.
- Accept, decline, or reschedule bookings.
- Host private sessions.
- Create session summaries.
- Create private notes where legally permitted.
- Share resources.
- Create and manage support groups.
- View only authorized client information.

Group moderator

Can:

- Create and schedule groups.
- Approve membership.
- Manage waiting-room participants.
- Mute or remove participants.
- Lock rooms.
- Manage chat and reports.

Administrator

Can:

- Review counselor credentials.
- Suspend or restore accounts.
- Review reports.

- Manage categories and emergency contacts.
- View operational analytics.
- Manage retention policies.
- Inspect security audit events.
- Never casually browse private counseling notes.

Use a separate, logged “break-glass” permission for exceptional authorized access to sensitive records.

6. Required pages

Build all of the following pages.

Public pages

- Landing page.
- How It Works.
- Counselor Directory.
- Counselor Details.
- Public Groups.
- Group Details.
- Resource Library.
- About.
- Contact.
- Help Centre.
- Safety and Emergency Information.
- Privacy Policy.
- Terms of Service.
- Counselor Code of Conduct.
- Accessibility Statement.

Authentication pages

- Register.
- Sign In.
- Email Verification.
- Forgot Password.
- Reset Password.
- Multi-factor Authentication.
- Device and Session Management.

Participant dashboard

- Dashboard Overview.
- Find a Counselor.
- Upcoming Sessions.
- Past Sessions.
- Calendar.
- Groups.

- Messages.
- Saved Resources.
- Notifications.
- Profile.
- Privacy and Consent.
- Data Export.
- Account Deletion.

Counselor dashboard

- Overview.
- Verification Status.
- Professional Profile.
- Credentials.
- Availability.
- Booking Requests.
- Upcoming Sessions.
- Session History.
- Client-visible Summaries.
- Private Notes.
- Messages.
- Groups.
- Resources.
- Reports.
- Account Settings.

Meeting pages

- Device Check.
- Waiting Room.
- Active Meeting.
- Reconnection Screen.
- Session Ended.
- Session Feedback.

Administrator dashboard

- Operational Overview.
- User Management.
- Counselor Verification.
- Credential Review.
- Group Management.
- Session Metadata.
- Safety Reports.
- Resource Moderation.
- Emergency Contacts.
- Audit Logs.
- Notification Templates.

- Privacy Requests.
- Retention Settings.
- Platform Settings.
- System Health.

Every page must have real routing, loading states, empty states, error states, and permission handling.

7. Design requirements

Create a calm, professional, non-stigmatizing interface.

Use:

- Deep navy for structure.
- Calm teal as the primary accent.
- Warm neutral backgrounds.
- Green only for positive or connected states.
- Amber for warnings.
- Red only for destructive actions, danger, and emergencies.

Requirements:

- Clear typography.
- Large touch targets.
- Keyboard navigation.
- Visible focus indicators.
- WCAG 2.2 AA contrast.
- Screen-reader labels.
- Reduced-motion support.
- Mobile-first responsive layout.
- Skeleton loaders.
- Helpful empty states.
- Confirmation dialogs for destructive actions.
- Toasts for completed background actions.

Do not create decorative controls that do nothing.

8. Authentication and account security

Implement:

- Email and password registration.
- Email verification.
- Secure password reset.
- Argon2id password hashing.
- Optional Google and Microsoft OAuth adapters.
- MFA for counselors and administrators.

- Secure HttpOnly cookies.
- Short session lifetime for sensitive dashboards.
- Refresh-token rotation where tokens are used.
- Active-device list.
- Remote session revocation.
- Brute-force protection.
- Login rate limiting.
- Suspicious-login notification.
- Account lock and recovery workflow.

Never store authentication tokens in browser local storage.

9. Counselor verification

Counselors must provide:

- Full legal name.
- Public display name.
- Profile photograph.
- Professional category.
- Biography.
- Languages.
- Areas of support.
- Country and service region.
- Registration or license number when applicable.
- Issuing institution.
- Credential documents.
- Expiration date.
- Years of experience.
- Acceptance of the counselor code of conduct.

Verification states:

- Draft.
- Submitted.
- Under Review.
- Additional Information Required.
- Approved.
- Rejected.
- Suspended.
- Expired.

Administrators must record a reason and audit event for every decision.

Never expose private credential documents publicly.

10. Counselor search and discovery

Allow users to filter counselors by:

- Professional category.
- Verified status.
- Support topic.
- Language.
- Country or region.
- Available date.
- Voice or video availability.
- Individual or group counseling.
- Accessibility support.

Display:

- Photograph.
- Name.
- Verification badge.
- Professional category.
- Languages.
- Support topics.
- Next available time.
- Short biography.
- Profile button.
- Booking button.

Do not rank counselors using hidden discriminatory criteria.

11. Scheduling system

Counselors can define:

- Weekly availability.
- Date-specific availability.
- Breaks.
- Time off.
- Minimum booking notice.
- Maximum booking window.
- Session duration.
- Buffer time.
- Participant capacity.
- Supported meeting type.

Booking workflow:

1. User selects a counselor.

2. User selects an available slot.
3. User selects voice or video.
4. User provides the minimum necessary booking reason.
5. User accepts privacy and session conditions.
6. Backend locks the slot in a transaction.
7. Counselor automatically accepts or manually reviews it.
8. Notifications are queued.
9. Calendar entries are generated.
10. A meeting room is created shortly before the session.

Prevent:

- Double booking.
- Booking expired slots.
- Time-zone mistakes.
- Unauthorized rescheduling.
- Duplicate requests.

Store all timestamps in UTC and display them in the user's selected time zone.

12. Video and voice meeting system

Use LiveKit as the media layer.

The backend must generate short-lived room tokens. Never generate privileged LiveKit tokens in the browser.

Implement:

- One-to-one rooms.
- Group rooms.
- Voice-only mode.
- Camera on or off.
- Microphone mute and unmute.
- Device selection.
- Speaker selection where supported.
- Camera switching on mobile.
- Screen sharing.
- Participant grid.
- Active-speaker view.
- Participant list.
- Waiting room.
- Hand raising.
- In-call chat.
- Host mute.
- Remove participant.
- Lock room.

- End meeting for everyone.
- Leave meeting.
- Network-quality indicator.
- Automatic reconnection.
- Low-bandwidth mode.
- Audio fallback when video becomes unstable.
- Permission-denied guidance.
- Device test before joining.

Room permissions must come from the backend and reflect the user's booking and role.

Do not allow a user to join a room by guessing its ID.

13. Session lifecycle

Use these states:

- Requested.
- Awaiting Approval.
- Confirmed.
- Rescheduled.
- Cancelled.
- Ready.
- Waiting.
- In Progress.
- Completed.
- No Show.
- Failed.

Enforce valid transitions on the backend.

Example:

Confirmed → Ready → Waiting → In Progress → Completed

A client cannot mark a session completed before it occurs. A counselor cannot silently change a completed session.

14. Group discussions

Support:

- Public discoverable groups.
- Private invite-only groups.
- Approval-required groups.
- One-time groups.
- Recurring groups.

- Maximum participant count.
- Group rules.
- Named moderators.
- Waiting room.
- Member removal and banning.
- Hand raising.
- Moderated chat.
- Anonymous display-name option where legally appropriate.
- Attendance metadata.
- Group-specific resources.

Do not expose group membership publicly unless a member explicitly consents.

Group meeting links must not function as permanent public authorization tokens.

15. Messaging

Implement secure platform messaging.

Features:

- One-to-one counselor conversations.
- Group announcements.
- Text messages.
- Approved file attachments.
- Read state.
- Message reporting.
- Conversation archiving.
- Block controls where appropriate.
- Notification preferences.

Apply:

- MIME validation.
- File-size limits.
- Malware-scanning adapter.
- Rate limits.
- Authorization on every download.
- Expiring signed file URLs.

Do not include message contents in ordinary application logs.

16. Notes and summaries

Separate:

Client-visible session summary

May include:

- Topics discussed.
- Agreed next steps.
- Shared resources.
- Follow-up date.

Private counselor notes

Requirements:

- Separate authorization rules.
- Encrypted sensitive fields.
- Access auditing.
- Configurable retention.
- No group-wide private notes containing unnecessary participant details.
- No administrator browsing without exceptional authorization.

The counselor must review and intentionally save every note.

17. Recording and transcription

Recording must be disabled by default.

Before recording:

1. Explain what will be recorded.
2. Explain the purpose.
3. Explain retention time.
4. Identify who can access it.
5. Obtain affirmative consent from every participant.
6. Display a persistent recording indicator.
7. Log the consent version and timestamp.

If all required participants do not consent, recording must remain disabled.

Provide two room configurations:

- End-to-end encrypted session without server-side recording or transcription.
- Standard secure media session where consented server-side recording is permitted.

Never imply that both server-side recording and strict end-to-end encryption are simultaneously available when the chosen media architecture does not support that combination.

18. Safety system

Include:

- Report user.
- Report message.
- Report group.
- Report counselor.
- Report unsafe session.
- Block user.
- Leave group.
- Emergency-information button.
- Country-specific emergency resource directory.
- Immediate-danger notice.
- Safeguarding workflow for minors.
- Escalation status for administrators.
- Evidence-preservation policy with strict access control.

Report states:

- Submitted.
- Triageed.
- Under Review.
- Action Required.
- Resolved.
- Dismissed.
- Escalated.

Never promise continuous human monitoring unless the service actually provides it.

19. Minors and vulnerable users

Do not assume ordinary adult consent is sufficient for minors.

Implement configurable rules for:

- Minimum user age.
- Guardian consent.
- School-managed accounts.
- Counselor authorization to work with minors.
- Group-age restrictions.
- Safeguarding contacts.
- Mandatory-reporting workflows defined by jurisdiction.
- Restricted direct messaging.
- Preventing unauthorized adult-minor contact.

Keep minor accounts disabled until the organization has selected and legally reviewed its operating rules.

20. Privacy and data controls

Implement privacy by design.

Users must be able to:

- View their profile data.
- Correct their data.
- Download permitted data.
- Review consent records.
- Withdraw optional consent.
- Request deletion.
- Manage notification preferences.
- Review active devices.
- Understand retention periods.

The system must support:

- Data minimization.
- Purpose limitation.
- Retention schedules.
- Encryption in transit.
- Encryption at rest.
- Field encryption for highly sensitive content.
- Pseudonymized analytics.
- Data-processing audit records.
- Data-breach workflow.
- Data-protection impact assessment documentation.
- Configurable regional storage.
- Cross-border transfer controls.

Do not collect counseling details merely for analytics.

21. Core database models

Create properly indexed Prisma models for:

- User.
- UserProfile.
- DeviceSession.
- Role.
- Permission.
- UserRole.
- CounselorProfile.
- CounselorCredential.
- CredentialReview.
- AvailabilityRule.

- AvailabilityException.
- Booking.
- Session.
- SessionParticipant.
- MeetingRoom.
- Group.
- GroupMember.
- GroupSchedule.
- Conversation.
- ConversationMember.
- Message.
- MessageAttachment.
- ConsentRecord.
- SessionSummary.
- PrivateCounselorNote.
- Recording.
- Resource.
- ResourceCategory.
- Notification.
- NotificationPreference.
- SafetyReport.
- ReportAction.
- EmergencyContact.
- PrivacyRequest.
- AuditLog.
- Organization.
- OrganizationMember.

Add:

- Foreign keys.
- Unique constraints.
- Soft deletion only where appropriate.
- Optimistic locking for frequently edited records.
- Created and updated timestamps.
- Explicit retention fields.
- Tenant or organization boundaries where applicable.

Never rely on frontend checks as database security.

22. API groups

Implement versioned endpoints for:

- `/auth`
- `/users`
- `/profiles`

- /counselors
- /credentials
- /availability
- /bookings
- /sessions
- /meetings
- /meetings/token
- /groups
- /conversations
- /messages
- /notes
- /recordings
- /consents
- /resources
- /notifications
- /reports
- /privacy
- /organizations
- /admin
- /health
- /ready

Every endpoint must include:

- Authentication requirements.
- Authorization checks.
- Request validation.
- Consistent error responses.
- Rate limiting where necessary.
- Audit logging for sensitive actions.
- OpenAPI documentation.
- Tests for successful and forbidden requests.

23. Notifications

Support:

- In-app notifications.
- Email reminders.
- Optional SMS adapter.
- Optional WhatsApp adapter.
- Browser notifications with permission.

Send notifications for:

- Email verification.
- Booking request.

- Booking confirmation.
- Declined booking.
- Rescheduling.
- Cancellation.
- Upcoming session.
- Counselor joining.
- Group invitation.
- New message.
- Credential decision.
- Report update.
- Privacy request update.
- Suspicious login.

Avoid including sensitive counseling information in email or SMS content.

24. Administrator dashboard

The dashboard must show:

- Total active users.
- Verified counselors.
- Pending credential reviews.
- Upcoming sessions.
- Active meeting rooms.
- Failed reminder jobs.
- Open safety reports.
- Privacy requests.
- System health.
- Media-service health.
- Database and queue health.

Analytics must use aggregated data and avoid exposing counseling content.

25. Security requirements

Implement:

- TLS in production.
- Strong Content Security Policy.
- CSRF protection.
- Secure cookie settings.
- CORS allowlist.
- Request-size limits.
- API rate limiting.
- Input validation.
- Output encoding.
- Parameterized database access.

- File validation.
- Secret rotation support.
- Database backups.
- Restore testing.
- Dependency scanning.
- Container scanning.
- Audit trails.
- Protection against insecure direct object references.
- Least-privilege service accounts.
- Separate development, staging, and production secrets.

Never log:

- Passwords.
- Authentication tokens.
- Meeting tokens.
- Complete counseling messages.
- Private notes.
- Recording contents.
- Encryption keys.

26. Error handling

Create useful handling for:

- Camera permission denied.
- Microphone permission denied.
- No device found.
- Device already in use.
- Slow internet.
- Disconnected call.
- Expired meeting token.
- Unauthorized room access.
- Full group.
- Counselor unavailable.
- Double-booked slot.
- Failed notification.
- Upload rejected.
- Server unavailable.
- Recording failure.

Errors must explain what happened and provide a safe recovery action.

27. Low-bandwidth operation

Optimize for users with limited or unstable internet.

Provide:

- Voice-only joining.
- Camera-off default preference.
- Automatic video-quality reduction.
- Simulcast or adaptive streaming.
- Audio-first reconnection.
- Lazy loading.
- Compressed images.
- Small JavaScript bundles.
- PWA-ready application shell.
- Clear network indicator.
- Retry without losing form data.

Test the website under simulated slow mobile networks.

28. Development environment

Provide `docker-compose.yml` containing:

- PostgreSQL.
- Redis.
- LiveKit.
- MinIO.
- Mailpit.
- API.
- Worker.
- Web application.

Provide:

- `.env.example`.
- Environment validation.
- Database migrations.
- Seed command.
- Demo development accounts.
- Health checks.
- Persistent named volumes.
- Safe container restart policies.

Required commands:

- `pnpm install`
- `pnpm setup`
- `pnpm dev`
- `pnpm test`
- `pnpm test:e2e`

- `pnpm lint`
- `pnpm typecheck`
- `pnpm build`
- `docker compose up --build`

Do not commit real secrets.

29. Seed data

Create safe fictional development data:

- One administrator.
- Two verified counselors.
- One unverified counselor.
- Three participants.
- One private group.
- One public group.
- Several availability slots.
- Upcoming and completed bookings.
- Sample public resources.
- Sample notification templates.

Clearly mark all seeded people as fictional.

30. Testing requirements

Create tests for:

- Registration and verification.
- Authentication.
- Password reset.
- MFA.
- Counselor credential submission.
- Administrator approval.
- Counselor search.
- Availability calculation.
- Concurrent booking protection.
- Booking cancellation.
- Meeting-token authorization.
- Waiting-room admission.
- Group capacity.
- Role permissions.
- Messaging authorization.
- File-download authorization.
- Consent collection.
- Recording blocked without consent.
- Private-note isolation.

- Safety-report workflow.
- Data export.
- Account deletion.
- Mobile layout.
- Keyboard navigation.
- Low-bandwidth reconnection.

Use fake camera and microphone devices in automated browser tests.

31. CI pipeline

On every pull request, run:

1. Dependency installation from the lockfile.
2. Formatting check.
3. Linting.
4. Type checking.
5. Unit tests.
6. API integration tests.
7. Database migration validation.
8. Production build.
9. End-to-end smoke tests.
10. Dependency vulnerability scan.
11. Container scan.

Do not deploy when required checks fail.

32. Production deployment

Support two media configurations:

Recommended first deployment

- Managed LiveKit Cloud.
- Managed PostgreSQL.
- Managed Redis.
- S3-compatible object storage.
- Containerized web, API, and worker services.
- CDN and Web Application Firewall.

Self-hosted deployment

- LiveKit server.
- TURN/TLS support.
- Public domain.
- Trusted SSL certificates.
- Correct UDP and TCP firewall ports.

- Redis for distributed LiveKit nodes.
- Monitoring and autoscaling.
- Tested recording and storage workers.

Include deployment documentation for both configurations.

33. Required documentation

Create:

- `README.md`.
- Local setup guide.
- Architecture overview.
- Database guide.
- API guide.
- LiveKit configuration guide.
- Counselor verification guide.
- Privacy and retention guide.
- Safeguarding configuration guide.
- Administrator manual.
- Testing guide.
- Backup and restore guide.
- Production deployment guide.
- Incident-response guide.

34. No-placeholder rule

Do not leave:

- Dead buttons.
- Fake dashboards.
- Hardcoded success messages.
- Fake call controls.
- Empty links.
- Unimplemented forms.
- TODO-only endpoints.
- Client-only authorization.
- Mock data in production paths.

If an external provider is not configured, show a clear “provider not configured” state and provide a working development adapter such as Mailpit or an in-app notification outbox.

35. Definition of done

The project is complete only when:

- A participant can register and verify their email.

- A counselor can submit credentials.
- An administrator can approve the counselor.
- The counselor can create availability.
- A participant can book an available slot.
- Both receive reminders.
- Both can run a device test.
- Only authorized participants can enter the room.
- They can conduct a voice or video session.
- The call reconnects after a temporary network loss.
- The counselor can save a permitted summary.
- The participant can view that summary.
- A moderator can host and manage a group session.
- Recording cannot start without required consent.
- Users can submit safety and privacy requests.
- Administrators can process those requests.
- Every visible control has a tested action.
- All automated tests pass.
- The application builds successfully.
- Docker Compose starts the local system.
- Production setup and security limitations are documented.

36. Final implementation instruction

Begin by producing:

1. Architecture decision record.
2. Database relationship plan.
3. Route and page inventory.
4. Permission matrix.
5. API contract.
6. Repository scaffold.
7. Database migrations.
8. Authentication.
9. Scheduling.
10. Video and voice meetings.
11. Messaging and groups.
12. Safety and privacy controls.
13. Administrator tools.
14. Automated tests.
15. Docker deployment.
16. Production documentation.

After each stage:

- Run linting.
- Run type checking.
- Run relevant tests.

- Fix errors before continuing.
- Show the changed files.
- State any remaining external credentials or infrastructure requirements.

Do not redesign completed UI sections without a functional reason. Do not claim production readiness until the defined tests and security checks pass.